

DESARROLLO DE APLICACIONES MULTIPLATAFORMA

CURSO 2

AUTOR:
ANTONIO CANO RUIZ

MÓDULO:
PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES

UT6 - La Forja Del Servidor Central

AUTOR: Antonio Cano Ruiz

Paso 1: Configuración de Dependencias y Permisos **3**
Paso 1.1: Permiso de Red 3
Paso 1.2: Dependencia de Media3 ExoPlayer 3
Paso 2: Recurso de Audio **4**
Paso 3: Diseño de la Interfaz (activity_laboratorio3.xml) **4**
Paso 4: Lógica de Control **5**
Paso 4.1: Inicializar ExoPlayer 6
Paso 4.2: Configurar Spinner 7

Paso 1: Configuración de Dependencias y Permisos

Paso 1.1: Permiso de Red

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
        xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.INTERNET" />

    <application
            android:allowBackup="true"
            android:dataExtractionRules="@xml/data_extraction_rules"
            android:fullBackupContent="@xml/backup_rules"
            android:icon="@mipmap/ic_launcher"
            android:label="La Forja del Servidor Central"
            android:roundIcon="@mipmap/ic_launcher_round"
            android:supportsRtl="true"
            android:theme="@style/Theme.LaForjaDelServidorCentral">
        <activity
                android:name=".MainActivity"
                android:exported="true"
                android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
```

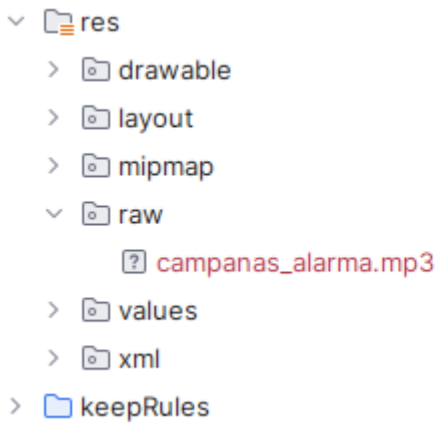
Se añade el permiso de ***Internet***

Paso 1.2: Dependencia de Media3 ExoPlayer

```
dependencies {
    implementation(libs.androidx.activity.ktx) androidx.activity:activity-ktx:1.13.0
    implementation(libs.androidx.appcompat) androidx.appcompat:appcompat:1.8.0
    implementation(libs.androidx.constraintlayout) androidx.constraintlayout:constraintlayout:2.2.2
    implementation(libs.androidx.core.ktx) androidx.core:core-ktx:1.19.1
    implementation(libs.material) com.google.android.material:material:1.14.0
    testImplementation(libs.junit) junit:junit:4.13.2
    androidTestImplementation(libs.androidx.espresso.core) androidx.test.espresso:espresso-core:3.7.0
    androidTestImplementation(libs.androidx.junit) androidx.test.ext:junit:1.3.0
    implementation("androidx.media3:media3-exoplayer:1.4.1")
    implementation("androidx.media3:media3-ui:1.4.1")
    implementation("androidx.media3:media3-common:1.4.1")
}
```

Se añaden las dependencias de ***Media3 Exoplayer***

Paso 2: Recurso de Audio



Se añade a la carpeta **raw** el archivo .mp3 con el sonido de las campanas

Paso 3: Diseño de la Interfaz (activity_laboratorio3.xml)

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="16dp"
        android:background="#7561"
        xmlns:app="http://schemas.android.com/apk/res-auto"
        android:id="@+id/main">

    <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Puesto de Transmisión:"
            android:textColor="#FFFFFF"
            android:textStyle="bold"
            android:textSize="16sp"
            android:layout_marginBottom="8dp" />
    <Spinner
            android:id="@+id/spinnerCanales"
            android:layout_width="match_parent"
            android:layout_height="48dp"
            android:background="#7561"
            android:padding="8dp"
            android:layout_marginBottom="16dp"/>
```

Se crea un TextView con el texto ***"Puesto de Transmisión:"***, y a continuación se añade el Spinner que se va a rellenar más tarde en el ActivityMain con un ArrayAdapter,

```
26
27      <androidx.media3.ui.PlayerView
28              android:id="@+id/playerView"
29              android:layout_width="match_parent"
30              android:layout_height="0dp"
31              android:layout_weight="1"
32              android:background="#000000"
33              app:use_controller="true"
34              app:show_timeout="5000"/>
35
36      <TextView
37              android:id="@+id/txtEstadoConexion"
38              android:layout_width="match_parent"
39              android:layout_height="wrap_content"
40              android:layout_marginTop="16dp"
41              android:gravity="center"
42              android:text="Seleccione un canal..."
43              android:textColor="#00E5FF"
44              android:textSize="18sp"
45              android:textStyle="bold" />
46
47  </LinearLayout>
```

Al final se añade un PlayerView, en el que se va a mostrar el video, y un TextView donde va a poner el Estado de la Conexión del vídeo

Paso 4: Lógica de Control

```
2 Usages
22  class MainActivity : AppCompatActivity() {
23
        3 Usages
24      private lateinit var spinnerCanales: Spinner
        2 Usages
25      private lateinit var playerView: PlayerView
        5 Usages
26      private lateinit var txtEstadoConexion: TextView
27
        5 Usages
28      private var exoPlayer: ExoPlayer? = null
        6 Usages
29      private var mediaPlayer: MediaPlayer? = null
30
        2 Usages
31      private val canalesMap = mapOf(
32          "Minas Tirith" to "https://raw.githubusercontent.com/mediaelement/mediaelement-files/master/big_buck_bunny.mp4",
33          "Isengard" to "https://raw.githubusercontent.com/mediaelement/mediaelement-files/master/echo-hereweare.mp4",
34          "Rivendell" to "https://raw.githubusercontent.com/intel-iot-devkit/sample-videos/master/person-bicycle-car-detection.mp4"
35      )
36
```

Primero creamos las variables que vamos a usar en toda la Activity, y creamos un **Map** con los nombres de los puestos, y una **url** a un vídeo (Son aleatorios de github).

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContentView(R.layout.actuvity_laboratorio3)

    spinnerCanales = findViewById(R.id.spinnerCanales)
    playerView = findViewById(R.id.playerView)
    txtEstadoConexion = findViewById(R.id.txtEstadoConexion)


    inicializarExoPlayer()
    configurarSpinner()

    ViewCompat.setOnApplyWindowInsetsListener( view = findViewById(R.id.main)) { v, insets ->
        val systemBars = insets.getInsets( typeMask = WindowInsetsCompat.Type.systemBars())
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
        insets
    }
}

1 Usage
```

En la función **onCreate,** se cargan el Spinner, el PlayerView y el TextView, y se traen de la vista.

Paso 4.1: Inicializar ExoPlayer

```
private fun inicializarExoPlayer() {
    exoPlayer = ExoPlayer.Builder( context = this).build().also { player ->
        playerView.player = player
        // Listener para monitorizar mutaciones del buffer y estados del reproductor
        player.addListener( listener = object : Player.Listener {
            override fun onPlaybackStateChanged(playbackState: Int) {
                when (playbackState) {
                    Player.STATE_BUFFERING -> {
                        txtEstadoConexion.text = "Cargando Búfer..."
                    }
                    Player.STATE_READY -> {
                        txtEstadoConexion.text = "Transmitiendo en vivo"
                    }
                    Player.STATE_ENDED -> {
                        txtEstadoConexion.text = "Conexión terminada"
                        reproducirAlarmaAcustica()
                    }
                    Player.STATE_IDLE -> {
                        txtEstadoConexion.text = "Conexión inactiva"
                    }
                }
            }
        })
    }
}
```

Se construye un objeto ExoPlayer, que va a tener dentro un Listener, para saber cuando está cargando, reproduciendo, terminado o sin conexión.
Cuando detecta cualquiera de esos estados cambia el texto del TextView de la vista.
Además cuando el video está en estado **“STATE_ENDED”**, se llama a una función que reproduce unas campanas

```
15      private fun reproducirAlarmaAcustica() {
16          // Liberar instancia anterior si la hubiera
17          mediaPlayer?.release()
18          mediaPlayer = null
19          // Instanciar y disparar de forma concurrente el MediaPlayer con el recurso físico local
20          mediaPlayer = MediaPlayer.create( context = this,  resid = R.raw.campanas_alarma)?.apply {
21              // Liberación automática al terminar la reproducción del audio
22              setOnCompletionListener { mp ->
23                  mp.release()
24                  mediaPlayer = null
25              }
26              start()
27          }
28      }
```

Esta función libera el mediaPlayer que lo tenía ocupado el del vídeo, y se lo asigna a otro que va a reproducir el audio. Con la función **setOnCompletionListener** podemos ejecutar código cuando termine la reproducción, lo he usado para hacer que se libere el mediaPlayer y pase a null.

Paso 4.2: Configurar Spinner

```
83      private fun configurarSpinner() {
84          val nombresCanales = canalesMap.keys.toList()
85          val adapter = ArrayAdapter(
86              context = this,
87              resource = android.R.layout.simple_spinner_item,
88              objects = nombresCanales
89          ).apply {
90              setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)
91          }
92          spinnerCanales.adapter = adapter
93          // Listener de selección de canal en tiempo real
94          spinnerCanales.onItemSelectedListener = object : AdapterView.OnItemSelectedListener {
95              override fun onItemSelected(parent: AdapterView<*>?, view: View?, position: Int, id: Long) {
96                  val canalSeleccionado = nombresCanales[position]
97                  val url = canalesMap[canalSeleccionado]
98                  if (!url.isNullOrEmpty()) {
99                      sintonizarCanal(url)
100                 }
101             }
102             override fun onNothingSelected(parent: AdapterView<*>?) {}
103         }
104     }
105
```

Esta función va a cargar los valores que van a haber en el Spinner y se va a encargar de la lógica de cuando se pulsa.
Se obtienes los canales, que son las claves del mapa que hemos creado al principio, y con ArrayAdapter, se meten como items del Spinner. Y a continuación se le asigna al Spinner ese adaptador.

Cuando se selecciona un item del Spinner, se obtiene el canal que se ha pulsado, con la posición, y a continuación la Url que tiene asignada.
A continuación se llama a la función **sintonizarCanal.**

```
1 Usage
106     private fun sintonizarCanal(url: String) {
107         exoPlayer?.let { player ->
108             val mediaItem = MediaItem.fromUri(url)
109             player.setMediaItem(mediaItem)
110             player.prepare()
111             player.play()
112         }
113     }
114
```

Esta función recibe la Url como parámetro, y la asignamos como el item que se va a reproducir. Como último paso preparamos el búfer y reproducimos.